

JavaScript

Language basics and the DOM API

Web Programming I

UEF// University of Eastern Finland

Course materials originally by Sonsoles López-Pernas, adapted by Maria Kuismin



JavaScript

Language basics and the DOM API

Student is able to
design and implement interactive and responsive web pages using HTML, CSS, JavaScript, and Bootstrap.

Content

Role of JavaScript

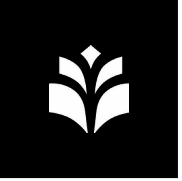
- What is JavaScript?
- Where does JavaScript run, and what does it do in the browser?

JavaScript basics

- How to declare variables and work with primitive and non-primitive types?
- How to control program flow with conditions, loops, and functions?

JavaScript in the browser: the DOM API

- How does JavaScript interact with the HTML page?
- How to listen to and react to user actions?



Role of JavaScript

- What is JavaScript?
- Where does JavaScript run, and what does it do in the browser?

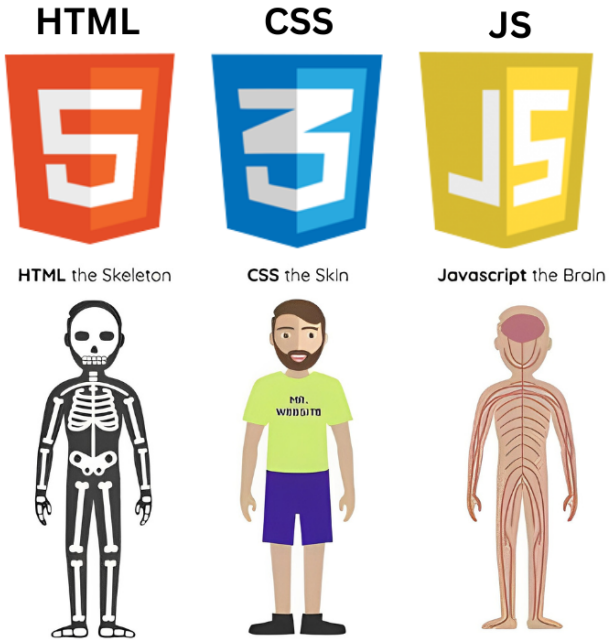
JavaScript basics

- How to declare variables and work with primitive and non-primitive types?
- How to control program flow with conditions, loops, and functions?

JavaScript in the browser: the DOM API

- How does JavaScript interact with the HTML page?
- How to listen to and react to user actions?

Client technologies




JAVA IS TO JAVASCRIPT
AS
CAR IS TO CARPET
Java
JS

Java vs. Javascript

Features	Java	JavaScript
Primary Use Case	• Server-side development, Android apps	• Front-end web development
Compilation	• Compiled into bytecode	• Interpreted at runtime
Syntax	• Strongly typed, compiled	• Loosely typed, interpreted
Platform	• Versatile, platform-independent	• Web browsers, primarily client-side
Development Role	• Backend development, large-scale system	• Front-end development, UI enhancement
Mobile App Development	• Yes, with a focus on Android apps	• Yes, with frameworks like React Native
Example Frameworks/ Libraries	• Spring, Hibernate	• React, Angular, Vue.js
Common IDEs	• Eclipse, IntelliJ IDEA	• Visual Studio Code, Sublime Text

* It can be used for backend development through node.js

JavaScript runs in two environments

In the browser (client-side)

- reacts to user actions (clicks, typing, scrolling)
- updates what you see on the page without reloading it
- examples: showing/hiding content, form validation, animations, updating a like counter

In the server (server-side)

- handles data that needs to be stored or kept secure
- processes business logic and communicates with databases
- examples: saving user accounts, processing payments, sending emails

Example: Chat application



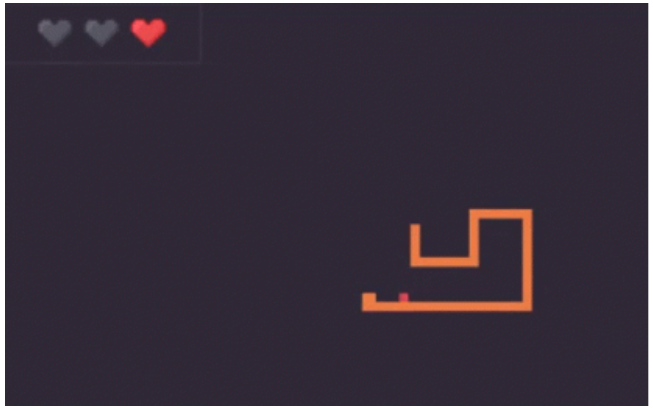


Chat application in the browser

- loads HTML, CSS, and JS from the website
- receives new messages from the server and shows them by inserting new HTML in the conversation area
- sends the user's typed messages to server and adds them to conversation area

All updates happens without reloading the page

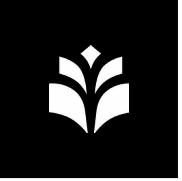
Example: Snake game



Game running in the browser

- loads HTML, CSS, and JS from the game website
- listens to the user's keyboard (arrow keys)
- updates the snake's direction and continuous movement
- updates points, lives, etc.

All this happens without reloading the page



What JavaScript can do in the browser

The browser gives JavaScript access to several APIs:

- DOM (Document Object Model) — modify the HTML page (reads, changes, creates, and removes HTML elements on the page) → focus of this course
- other browser APIs: Graphics (Canvas, WebGL), Audio/Video, Geolocation, Storage, Notifications, ...

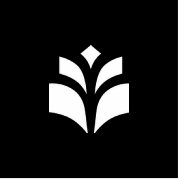
Adding JavaScript to an HTML page

```
<script src="path/to/your/file.js"></script>
```

...or...

```
<script>
# Write here your code

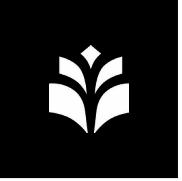
</script>
```



Role of JavaScript
- What is JavaScript?
- Where does JavaScript run, and what does it do in the browser?

JavaScript basics
- How to declare variables and work with primitive and non-primitive types?
- How to control program flow with conditions, loops, and functions?

JavaScript in the browser: the DOM API
- How does JavaScript interact with the HTML page?
- How to listen to and react to user actions?



Variables

Variables have a type, but we do not need to declare it (JS infers it)

```
var weight = 4.3
var hasShorthair = true


const hasShorthair = true; # Constant, cannot be changed
let weight = 4.3; # Same as var but with reduced scope,
accessible inside the block (curly braces)
```

Let and const are preferred over var, because reduced scope prevents accidental bugs

Primitive Types

Number: Represents both integer and floating-point numbers.

```
let age = 6; // Number
let weight = 4.3; // Floating-point number
```

String: Represents textual data. Strings are enclosed in quotes (' ', " ", or ` ` for template literals).

```
let name = "Toast"; // String
```

Boolean: Represents logical entities with two possible values: true or false.

```
let isSleeping = true; // Boolean
```

Undefined: A variable that has been declared but not assigned a value. The default value is undefined.

```
let owner; // Undefined
```

Null: Represents the intentional absence of any object value. It's a special type that represents an empty or non-existent reference.

```
let previousOwner = null; // Null
```

Non-zero value



0



null



undefined



Non-primitive types › Objects

collection of key-value pairs

Creating an object directly

```
let cat = {
  name: "Toast",
  age: 6,
  isSleeping: true
};
cat.name;          // "Toast"
cat.age;           // 6
cat.isSleeping;    // true
```

Creating an object from existing variables

```
let catName = "Toast";
let catAge = 6;

let cat = { catName: catName, catAge: catAge };

// Shorthand: if the key and variable name match
let cat = { catName, catAge };
// Same result: { catName: "Toast", catAge: 6 }
```

Non-primitive types › Array

Creating an array

```
let cats = ["Toast", "Naranja", "Kinsky"];
cats.length;      // 3
```

Accessing elements by index

```
cats[0];          // "Toast"
cats[2];          // "Kinsky"
```

Adding and removing elements

```
cats.push("Ramona");  // adds to end → ["Toast", "Naranja", "Kinsky", "Ramona"]
cats.pop(); // removes last → ["Toast", "Naranja", "Kinsky"]
```

Looping over an array

```
for (let cat of cats) {
  console.log(cat); // "Toast", "Naranja", "Kinsky"
}
```

Transforming with arrow functions

```
cats.map(cat => cat.toUpperCase()); // ["TOAST", "NARANJA", "KINSKY"]
cats.filter(cat => cat.length > 5); // ["Naranja", "Kinsky"]
```

Non-primitive types › Functions

A reusable block of code that can take input, process it, and return an output.

Traditional syntax

```
function greetCat(name) {
  return "Terveh, " + name;
}
```

Arrow function syntax (ES6)

```
const greetCat = (name) => {
  return "Terveh, " + name;
};
```

Arrow function shorthand

```
const greetCat = (name) => "Terveh, " +
name;
```

Calling the function

```
greetCat("Naranja");   // "Terveh, Naranja"
greetCat("Toast");  // "Terveh, Toast"
```

The arrow shorthand automatically returns the expression. With curly braces, you need an explicit return statement.

Functions: three common patterns

1. No input needed
— always does the same thing

```
const greet = () => "Terveh!";
greet();   // "Terveh!"
```

2. Uses an outer variable
— works, but tied to that variable

```
const catName = "Naranja";
const greetOurCat = () => "Terveh, " + catName;
greetOurCat();   // "Terveh, Naranja"
```

3. Takes a parameter
— flexible and reusable

```
const greetAnyCat = (name) => "Terveh, " + name;
greetAnyCat("Naranja");   // "Terveh, Naranja"
greetAnyCat("Toast");  // "Terveh, Toast"
```

Arrow functions appear often with arrays and event listeners.

```
cats.map(cat => cat.toUpperCase());

btn.addEventListener('click', () => {
  console.log('Liked!');
} );
```

Note: console.log() prints to the browser console.

Operators

Arithmetic Operators: +, -, *, /, % (modulus), ** (exponentiation).

```
let sum = 5 + 3;   // 8
let power = 2 ** 3; // 8
```

Assignment Operators: =, +=, -=, *=, /=.

```
let totalCats = 2;
totalCats += 3; // totalCats is now 5
```

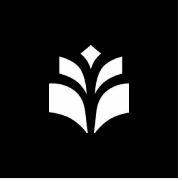
Comparison Operators: ==, ===, !=, !==, <, >, <=, >=.

```
let isEqual = (5 == "5");  // true (loose comparison)
let isStrictEqual = (5 === "5"); // false (strict comparison)
```

Logical Operators: && (AND), || (OR), ! (NOT).

```
let result = (true && false);  // false

let orResult = (true || false); // true
```



Control flow › Condiotional Statements

```
let catAge = 5;

if (catAge < 1) {

  console.log("Kitten");

} else if (catAge < 10) {

  console.log("Adult");

} else {

  console.log("Senior");

}
```

Control flow › Ternary operator

The ternary operator is useful when you want to assign a value based on a condition, all in one expression.

```
condition ? value_if_true : value_if_false
```

e.g.

```
{isLoggedIn ? "Welcome back!" :
"Please log in"}
```

```
// Ternary statement
let age = 6;
let ageGroup = (age >= 2) ? "Adult" :
"Young"; // Adult

// Full if/else statement
let age = 6;
let ageGroup;
if (age >= 2) {
  message = "Adult";
} else {
  message = "Young";
}
```

Control flow › Switch operator

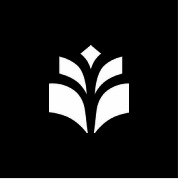
Switch is used for multiple condition checks

```
switch (day) {
  case 'Monday': return 'Start of the week';
  case 'Friday': return 'End of the week';
  default: return 'Middle of the week';
}
```

Alternative: object lookup (often cleaner)

```
const dayMessages = {
  'Monday': 'Start of the week',
  'Friday': 'End of the week',
};
return dayMessages[day] || 'Middle of the week';
// value of given day OR 'Middle of the week'
```

Both achieve the same result. Object lookup is common in modern JavaScript and React. Switch is still used, especially in reducers and event handlers with many cases.

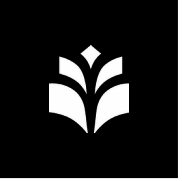


Loops › while

```
let i = 0;

while (i < 5) {
  console.log(i);  // Outputs 0 to 4
  i++;

}
```



Loops › for

```
for (let i = 0; i < 5; i++) {
  console.log(i);  // Outputs 0 to 4
}


let cats = ['Toast', 'Naranja', 'Kinsky'];
for (let cat of cats) {
  console.log(cat);
}

let cat = {name: 'Kinsky', age: 10};
for (let key in cat) {
  console.log(key, cat[key]);  // Outputs 'name', 'age'
}
```

Some Array methods

Finding elements

```
cats.find(cat => cat.length > 5); // "Naranja" (first match)
cats.findIndex(cat => cat.length > 5); // 1 (index of first match)
cats.includes("Toast"); // true
```

Checking conditions

```
cats.every(cat => cat.length > 3); // true (all match?)
cats.some(cat => cat === "Toast"); // true (any match?)
cats.some(cat => cat.startsWith("T")); // true
```

Transforming

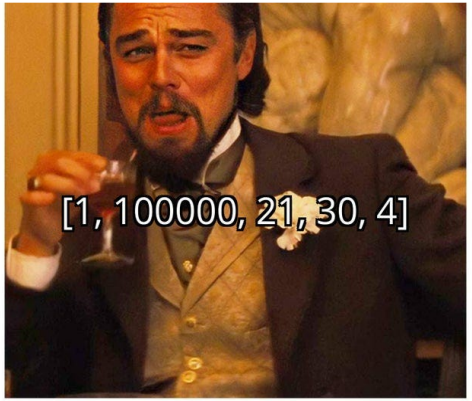
```
cats.join(", "); // "Toast, Naranja, Kinsky"
cats.reverse();  // ["Kinsky", "Naranja", "Toast"]
[...cats].sort(); // alphabetical sort (on a copy)
```

Aggregating

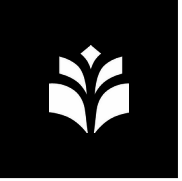
```
[3, 4, 5, 6].reduce((sum, n) => sum + n, 0);  // 18
```

People learning JavaScript:
”I’ll use array.sort() to sort this list of numbers!”

JavaScript:




```
let myNumbers = [21, 4, 30, 100000, 1];
myNumbers.sort((a, b) => a - b);
// [1, 4, 21, 30, 100000]
```



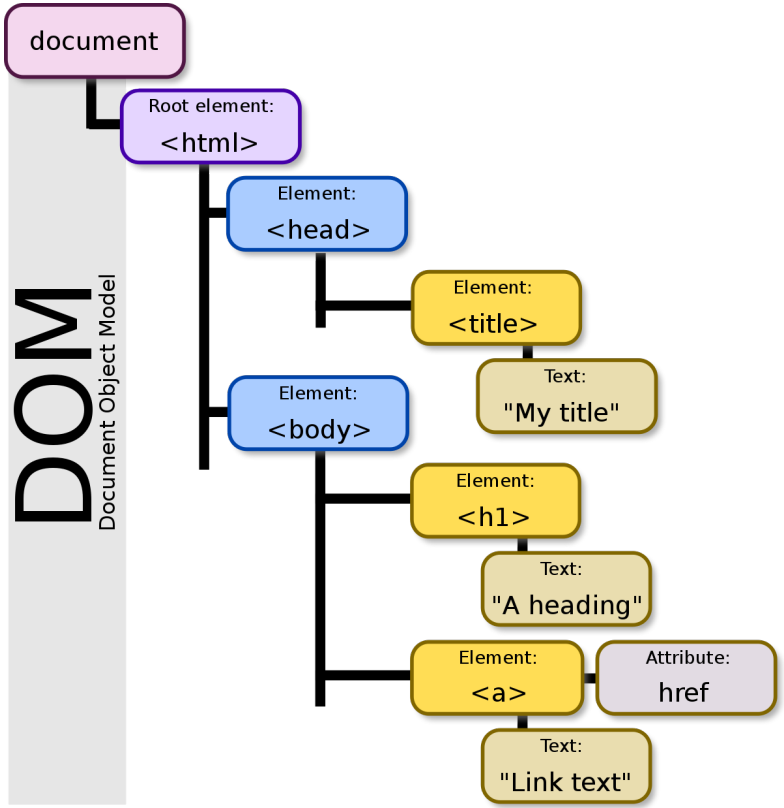
Role of JavaScript
- What is JavaScript?
- Where does JavaScript run, and what does it do in the browser?

JavaScript basics
- How to declare variables and work with primitive and non-primitive types?
- How to control program flow with conditions, loops, and functions?

JavaScript in the browser: the DOM API
- How does JavaScript interact with the HTML page?
- How to listen to and react to user actions?

Document Object Model

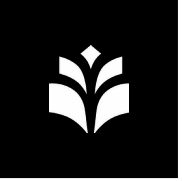
DOM (Document Object Model) is the JS API provided by the browser to manipulate the page's HTML



Selecting DOM Elements

To manipulate or read elements in the DOM, you first need to find/select the element you want to work with.

- **getElementById()**: Selects an element by its id attribute.
```
let container = document.getElementById('container');
```
- **getElementsByClassName()**: Selects all elements with a specific class name (returns an HTMLCollection).
```
let elements = document.getElementsByClassName('myClass');
```
- **getElementsByTagName()**: Selects all elements of a specific tag (returns an HTMLCollection).
```
let paragraphs = document.getElementsByTagName('p');
```
- **querySelector()**: Selects the first element that matches a CSS selector.
```
let heading = document.querySelector('nav > ul > .first');
```
- **querySelectorAll()**: Selects all elements that match a CSS selector (returns a NodeList).
```
let divs = document.querySelectorAll('div > p');
```



Manipulating DOM Elements › Changing content

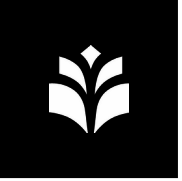
You can change/manipulate the element in three ways: changing **content**, attributes, or appearance

- **textContent:** Change or read the text inside an element.

```
document.getElementById('container').textContent =
 "New text content!";
```

- **innerHTML:** Change or read the HTML inside an element.

```
document.getElementById('container').innerHTML =
"<p>Updated HTML content</p>";
```



Manipulating DOM Elements › Attributes

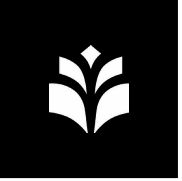
You can change/manipulate the element in three ways: changing content, **attributes**, or appearance

- **setAttribute():** Set or modify an attribute on an element.
```
document.querySelector('img').setAttribute('src', 'image.jpg');
```
- **getAttribute():** Retrieve the value of an attribute.
```
let src = document.querySelector('img').getAttribute('src');
```
- **removeAttribute():** Remove an attribute from an element.
```
document.querySelector('img').removeAttribute('alt');
```

Manipulating DOM Elements › CSS Classes and style attributes

You can change/manipulate the element in three ways: changing content, attributes, or appearance

- **classList.add()**: Add a class to an element.
```
document.getElementById('container').classList.add('newClass');
```
- **classList.remove()**: Remove a class from an element.
```
document.getElementById('container').classList.remove('oldClass');
```
- **classList.toggle()**: Toggle a class on or off.
```
document.getElementById('container').classList.toggle('active')
```
- **Modifying style**
```
document.querySelector('p').style.color = 'blue';
```



Manipulating DOM Elements › Creating DOM Elements

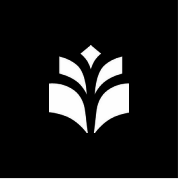
Finally, you can **add new elements** or remove existing ones

- **createElement**(): Creates a new HTML element.

```
let newDiv = document.createElement('div');
newDiv.textContent = "I am a new div!";
```

- **appendChild**(): Appends an element as the last child of another element.

```
document.body.appendChild(newDiv);
```



Manipulating DOM Elements › Removing DOM Elements

Finally, you can add new elements or **remove existing ones**

- **remove()**: Removes the selected element from the DOM.
```
document.getElementById('container').remove();
```
- **removeChild()**: Removes a child element from its parent.
```
let parent = document.getElementById('container');
let child = document.querySelector('p');
parent.removeChild(child);
```

Manipulating DOM Elements › Event listeners

Events are one of the most important aspects of the DOM, allowing you to make your web pages interactive. Common events include clicks, key presses, form submissions, etc.

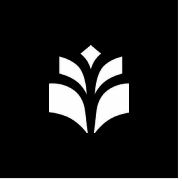
The element that we interact with

The type of interaction

```
document.getElementById('myButton').addEventListener('click', function() {
  alert('Button was clicked!');});
```

What happens after this interaction

https://www.w3schools.com/jsref/dom_obj_event.asp



Common events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user press a keyboard key
onload	The browser has finished loading the page

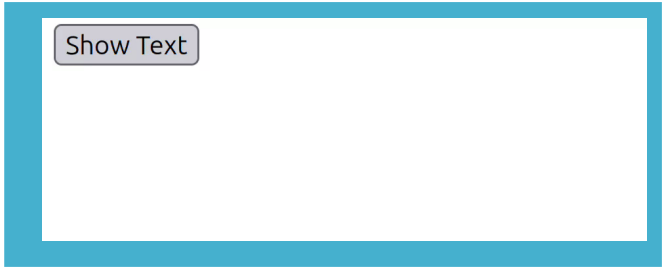
Example: Toggle button

```
<button id="toggleButton">Show Text</button>
<p id="textToToggle" style="display: none;">
Here is some hidden text that you can toggle on and off!</p>

<script>
    const button = document.getElementById('toggleButton');
    const text = document.getElementById('textToToggle');

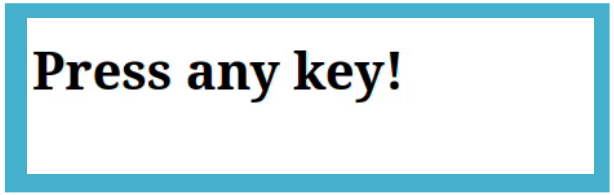
    // Event listener for button click
    button.addEventListener('click', () => {
        if (text.style.display === 'none') {
            text.style.display = 'block';
            button.textContent = 'Hide Text';
        } else {
            text.style.display = 'none';
            button.textContent = 'Show Text';
        }
    });
</script>
```





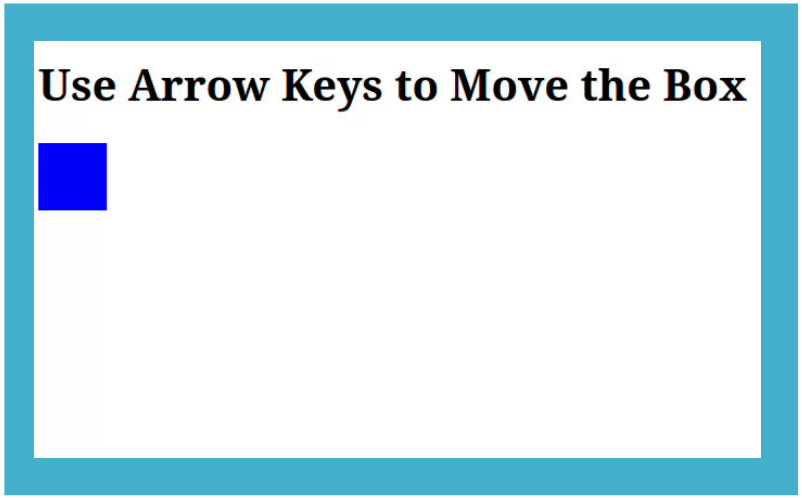
Example: Pressed key displayed

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Keypress Event</title>
</head>
<body>
    <h1 id="keyDisplay">Press any key!</h1>
    <script>
        const keyDisplay = document.getElementById('keyDisplay');
        // Event listener for keypress
        window.addEventListener('keydown', (event) => {
            keyDisplay.textContent = `You pressed: ${event.key}`;
        });
    </script>
</body>
</html>
```





Example: Moving box




```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Move Box with Arrow Keys</title>
    <style>
        #box {
            width: 50px;
            height: 50px;
            background-color: blue;
            position: relative;
            top: 0;
            left: 0;
        }
    </style>
</head>
<body>

    <h1>Use Arrow Keys to Move the Box</h1>
    <div id="box"></div>

    <script>
        const box = document.getElementById('box');
        let topPosition = 0;
        let leftPosition = 0;
        const moveAmount = 10;  // How much to move with each arrow press

        // Event listener for keydown event
        window.addEventListener('keydown', (event) => {
            switch(event.key) {
                case 'ArrowUp':
                    topPosition -= moveAmount;
                    break;
                case 'ArrowDown':
                    topPosition += moveAmount;
                    break;
                case 'ArrowLeft':
                    leftPosition -= moveAmount;
                    break;
                case 'ArrowRight':
                    leftPosition += moveAmount;
                    break;
            }
            // Apply the new positions
            box.style.top = topPosition + 'px';
            box.style.left = leftPosition + 'px';
        });
    </script>
</body>
</html>
```

Example: Chat application (conceptual)





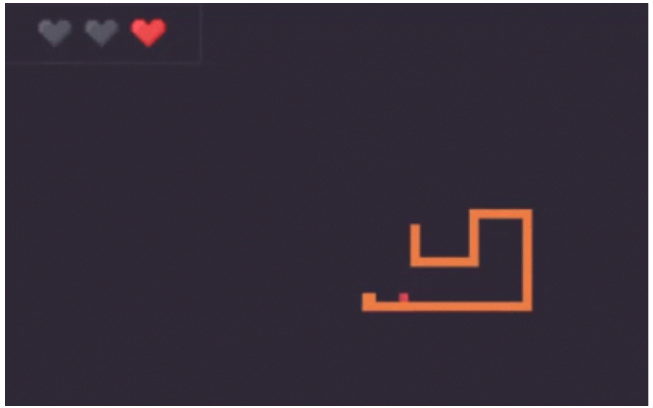
new incoming message → `createElement + textContent + appendChild`

sending a message → event listener on send button or `keydown` (Enter)

adding to the conversation → `appendChild` into the chat container

optionally styling messages → classList (e.g. `mine` v.s `theirs`)

Example: Snake game (conceptual)



keyboard input → `addEventListener('keydown',…)`

detect which arrow → `event.key`

move the snake → change element `'style'` (position)

update points and lives → `'textContent'`

Summary

- JavaScript is a language used to add interactivity to web pages.
- It runs in the browser, where it can modify HTML and respond to user actions.
- Variables (var, let, const) hold values of different types — numbers, strings, objects, arrays, or functions.
- Conditions, loops, and functions are used to control how the code behaves.
- The DOM API is the way JavaScript communicates with the HTML page — selecting, changing, creating, and removing elements.
- Event listeners detect user actions like clicks or key presses, making the page interactive.

Example: Kažigram with JavaScript

- const/let → storing form inputs and post data
- Array methods → managing the list of posts
- Form input handling → event listeners on submit
- `createElement + appendChild` → new post cards added to the page
- `classList.toggle` → like button (heart icon changes)
- `textContent` → like count updates
- `remove()` → delete post